1.库概念

```
是代码的二进制的封装形式
在其他的源代码或库中，可以直接调用库中的函数，但是看不到源代码
没有公开的源代码的

库的这种实现方式有利于模块化，只要接口合理的，不会影响库的使用
```

```
sum.c sum.h

int sum(int a,int b)
{
    return a+b;
}

xxx.c
    需要用到sum.c函数，原来的做法，直接把源代码拷贝一份
缺陷：
    1.直接给源代码，不利于知识产权的保护
      想给你用，但是又不想让你知道代码到底怎么写的
    2.假设有一天，sum.c的实现，更新换代，使用原来的sum的代码，需要重新拷贝，重新编译，打包
```

```
现在：使用库
sum.c sum.h

int sum(int a,int b)
{
    return a+b;
}
编译，生成libxxx.so(库的二进制文件)

xxx.c
    想要使用sum函数，直接链接到libxxx.so
    => a.out 运行a.out  加载libxxx.so就可以了
```

2.动态库

```
动态库的使用步骤
1.制作动态库
    a.编辑源代码(库的代码)
        eg:
            linkqueue.c 功能函数的实现
            linkqueue.h 接口文件
            bst.c
            bst.h
        注意：库的源代码 不能包含main函数
    b.生成动态库
        gcc(aarch64-linux-gnu-gcc) -shared -fpic -o libxxx.so 需要编译成库的.c文件
            -shared 生成动态库
            -fpic   与位置无关,相对路径
```

```
            -o      指定生成动态库的名字
                    动态库的名字 一般只以lib开头 以.so结尾
                    lib[0-9a-zA-Z]+\.so
        ubuntu:
            gcc -shared -fpic -o libbst.so bst.c linkqueue.c
2.使用动态库
    a.把头文件和库拷贝给需要的人

        gcc(aarch64-linux-gnu-gcc) xxx.c -I 目录1 -I 目录2 ... -L 目录名 -l 库名 -o
xxx
            -I  接一个目录，指定头文件的搜索路径，可以接多个
            -L  接一个目录，指定后面的哪个库的搜索路径
            -l  接库名，把lib .so去掉剩下的就是库名字

        eg:
            gcc main.c -I ./ -L ./ -l bst

            执行上面的命令，会生成可执行文件xxx,并且可执行文件xxx会用到库libxxx.so,但是在
生成xxx的时候，并没有把动态库的内容拷贝到xxx中，只是做了标记的功能
            ./main: error while loading shared libraries: libxxx.so: cannot open
shared object file: No such file or directory
            上面的错误的解决办法看b
      b.最终运行使用库的那个可执行文件前，确保依赖的库在运行的系统中，
        还需要让系统知道库在哪一个路径下

        方法1.修改环境变量(只适用LD_LIBRARY_PATH中有路径时)(ubuntu不能使用这种方法)
            export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:你依赖的路径(绝对路径)
        方法2.把依赖的库直接丢到/usr/lib目录下
```

练习:

```
制作一个二叉排序树的库，在开发板中运行库
    把库放到开发板
1.cd 进入到二叉排序树的目录下面
2.制作动态库
    aarch64-linux-gnu-gcc -shared -fpic -o libbst.so linkqueue.c bst.c
3.找个文件夹写main.c,main.c中需要调用库中的函数
    把 库的.h 和库拷贝到这个文件夹下面
4.编译(库文件的路径 头文件的路径都需要加入到编译的命令中)
    aarch64-linux-gnu-gcc main.c -I ./ -L ./ -l bst -o main
5.把main(可执行文件)和动态库下载到开发板
    使用rz -y命令下载到开发板 或 使用tftp
6.把动态库拷贝到开发板的/usr/lib目录下面
    cp libbst.so /usr/lib
7.运行
    ./可执行文件名
```

3.静态库

```
1.制作静态库
    1.编辑源代码
    2.把所有的源代码编译.o
    3.用ar命令把所有的.o文件打包，生成静态库
        (arm-linux-gnueabi-ar)ar -rcs libxxx.a 加上所有的.o文件名
            -r  更新
```

```
            -c   创建
            -s   索引
        把所有的.o打包生成静态库
        静态库叫libxxx.a 那么在用的时候就叫xxx

2.使用
    1.把静态库和头文件拷贝别人
        gcc(arm-linux-gnueabi-gcc) main.c -I ... -L ... -l xxx -o main
        注：main.c都需要自己单独写
    2.直接运行(在开发板中运行，下载可执行文件运行即可，不需要拷贝静态库)
```

练习:

```
把 不带头结点的单链表制作成 静态库，在开发板中运行
```

4.静态库和动态库的区别

```
动态库和静态库都是库，都是代码的二进制的封装。"可以让你用，但是让你不知道里面是怎么实现的"
区别：
    对于动态库，对使用动态库的代码，在编译时候，并没有把动态库编译进去
    仅仅做了一个标记：我的这个可执行程序在运行的时候，需要用到某某某动态库

    对于静态库，对使用静态库的代码 编译，把静态库的内容拷贝到可执行文件

    在运行的时候，对于动态而言，
        需要把动态库同时下载到目标板
    在运行的时候，对于静态而言，
        直接运行可执行程序，静态库不需要下载
```